TÁC VỤ VỚI TKQC

Tài liệu Phân tích Nghiệp vụ

Module: Tạo TKQC · Share TKQC · Kháng Nghị TKQC

Bối cảnh hệ thống	Hệ thống đã có: VIA store (DB), TKQC/BM/Page/Pixel theo từng VIA. Module này là bước tiếp theo: thực thi tác vụ hàng loạt trên TKQC.
Phạm vi tài liệu	3 sub-module: Tạo TKQC, Share TKQC (TDB — 1 VIA), Kháng Nghị TKQC. Bao gồm luồng nghiệp vụ, logic xử lý, trạng thái, API, DB schema bổ sung.
Không thuộc phạm vi	Pool Share (nhiều VIA song song) — sẽ là module riêng sau khi TDB hoàn thiện.
Người đọc	Frontend Dev (Vue 3), Backend Dev (Node.js), QA.

TỔNG QUAN MODULE TÁC VỤ VỚI TKQC

Vị trí trong hệ thống và nguyên tắc thiết kế

1. Vị trí trong hệ thống

Module này là tầng hành động — nằm trên tầng dữ liệu (VIA + tài sản đã có). Người dùng đã biết mình có những TKQC nào (từ module quản lý tài sản), module này cho phép thực thi hàng loạt tác vụ trên chúng.

Đã có (không thay đổi)	Module mới cần build
Bảng via_accounts — lưu VIA đã import	Sub-module Tạo TKQC hàng loạt
Bảng ad_accounts — TKQC theo từng VIA	Sub-module Share TKQC
Bảng business_managers — BM theo từng VIA	Sub-module Kháng Nghị TKQC bị disabled
Bảng pages, pixels — tài sản khác	Bảng jobs, job_logs, done_keys (DB mới)

2. Nguyên tắc thiết kế chung

- Mọi tác vụ đều chạy async — frontend không block chờ. Backend nhận config, tạo job, trả job ngay. Frontend poll status qua WebSocket hoặc HTTP.
- Mọi kết quả đều ghi log per-item (từng TKQC) — không chỉ tổng OK/fail.
- Job có thể dừng giữa chừng và Resume — nhờ done-set persist vào DB.
- TKQC status Disabled/Unavailable/Closed bị lọc bỏ tự động trước khi đưa vào bất kỳ tác vụ Share nào.

- Disabled: Tài khoản bị Facebook vô hiệu hóa — thường do vi phạm chính sách quảng cáo. Đây là trạng thái phổ biến nhất cần kháng nghị.

- Unavailable: Tài khoản không khả dụng — thường do lỗi hệ thống hoặc đang trong quá trình xử lý nội bộ của Facebook. Không thể thao tác.

- Closed: Tài khoản đã đóng vĩnh viễn — không thể phục hồi.

PHẦN 1 — TẠO TKQC

Tạo hàng loạt Tài Khoản Quảng Cáo trong nhiều BM

1.1 Mô tả nghiệp vụ

Người dùng chọn một VIA, chọn một hoặc nhiều BM thuộc VIA đó, cấu hình số lượng và template tên, rồi hệ thống tự động tạo TKQC trong từng BM. Sau khi tạo, TKQC có thể được giữ lại trong BM, xóa khỏi BM (tạo mồ côi), hoặc share ngay sang BM đích rồi xóa.

1.2 Điều kiện tiên quyết

- VIA đã import, status = "ok", token còn hạn.
- BM danh sách: lấy từ bảng business_managers WHERE via_id = viaId. Không cần fetch lại nếu đã có trong DB — chỉ fetch khi user chủ động refresh.
- Biết giới hạn BM (creationLimit) trước khi tạo — lấy từ bước Scan BM. Nếu chưa scan: hiện cảnh báo, cho phép tiếp tục nhưng có thể gặp lỗi "limit exceeded" giữa job.

1.3 Luồng người dùng

1. **Chọn VIA thực thi**
   Dropdown chọn 1 VIA từ danh sách via_accounts (status = "ok"). Hệ thống load danh sách BM của VIA đó từ DB.

2. **Chọn BM và scan giới hạn**
   Tick chọn 1 hoặc nhiều BM. Nút "Scan TKQC hiện có" gọi API scanBMDetail để lấy ownedCount, creationLimit, verification. BM đã full tự động disabled.

3. **Cấu hình tạo**
   Điền: số lượng mỗi BM, template tên, currency (chọn 1 lần, immutable sau khi tạo), timezone, chế độ sau khi tạo (Giữ / Đá / Share rồi đá).

4. **Preview và xác nhận**
   Hệ thống render trước danh sách tên TKQC sẽ được tạo. User xem, confirm số lượng. Hiện cảnh báo nếu currency/timezone chưa chọn.

5. **Chạy job**
   Backend tạo job, chạy async. Frontend poll WebSocket. Hiện progress bar + log realtime từng TKQC. Có thể dừng giữa chừng.

6. **Kết quả**
   Bảng kết quả: tên TKQC, BM tạo trong, actId, trạng thái (created / bm_full / failed / shared_removed). Nút export CSV.

1.4 Template tên

Template hỗ trợ 6 biến được replace tự động khi tạo mỗi TKQC:

Biến	Giá trị	Ví dụ
`{bm_name}`	Tên BM đang tạo trong	"Agency Alpha"
`{bm_id}`	ID của BM đang tạo trong	"823041xxx"
`{n}`	Số thứ tự tiếp nối từ ownedCount hiện tại. BM đang có 3 TKQC → batch này bắt đầu từ 4.	4, 5, 6...
`{i}`	Số thứ tự trong batch, luôn bắt đầu từ 1 bất kể BM đã có bao nhiêu.	1, 2, 3...
`{date}`	Ngày tạo dạng YYYY-MM-DD	"2026-04-01"
`{random}`	4 ký tự random alphanumeric, tái tạo mỗi lần	"k3x9"

1.5 Ba chế độ sau khi tạo

Chế độ	Chuỗi hành động	Khi nào dùng
Giữ lại	Tạo → done.	Tích lũy TKQC trong BM để dùng sau.
Đá khỏi BM	Tạo → DELETE khỏi BM nguồn.	TKQC mồ côi, chưa gắn BM nào. Phân phối sau.
Share rồi đá	Tạo → Share vào BM đích → DELETE khỏi BM nguồn. Nếu share fail: không DELETE, ghi status "created_share_fail".	Workflow tự động hóa: tạo xong phân phối ngay.

1.6 Logic engine (backend)

```
FOR mỗi bm IN selectedBMs:
  IF shouldStop: break

  FOR i = 1 TO countPerBM:
    IF shouldStop: break

    n = (bm.ownedCount || 0) + i
    name = resolveName(template, { bm, n, i, date, random })

    result = await createAdAccount({ bmId: bm.id, name, currency, timezoneId, token })
    // API: POST graph.facebook.com/v25.0/{bmId}/adaccount

    IF result.error.includes("limit"):
      log status = "bm_full"
      break  // skip remaining i cho BM này, tiếp BM tiếp theo

    actId = result.accountId

    IF mode == "shareAndRemove":
      FOR targetBM IN targetBMs:
        shareResult = await shareAdAccount(actId, targetBM.id, perms, token)
      IF any share failed:
        log status = "created_share_fail"
        continue  // không DELETE, tiếp TKQC tiếp theo

    IF mode IN ["remove", "shareAndRemove"] AND share OK:
      await removeAdAccount(bmId, actId, token)
      // API: DELETE {bmId}/owned_ad_accounts { adaccount_id }

    log status = created | removed | shared_removed
```

```
    sleep(delayMs)
```

1.7 Facebook API calls

Hành động	Endpoint	Fields/Body
Tạo TKQC	`POST /{bmId}/adaccount`	name, currency, timezone_id, end_advertiser:"NONE", media_agency:"NONE", partner:"NONE"
Xóa TKQC khỏi BM	`DELETE /{bmId}/owned_ad_accounts`	{ adaccount_id: actId }
Scan giới hạn BM	`GET /{bmId}/owned_ad_accounts`	fields: id,name,account_status — đếm count. Probe BM350: GET /{bmId}/owned_pixels.

1.8 Trạng thái kết quả per-TKQC

status	Mô tả
`created`	Tạo thành công, giữ trong BM.
`removed`	Tạo thành công, đã xóa khỏi BM nguồn.
`shared_removed`	Tạo + share thành công + đã xóa khỏi BM nguồn.
`created_share_fail`	Tạo OK nhưng share thất bại. Giữ nguyên trong BM nguồn để không mất.
`bm_full`	BM đạt giới hạn TKQC. Skip toàn bộ count còn lại của BM này.
`failed`	Lỗi tạo — xem error message.

1.9 DB Schema bổ sung

```
-- Job tạo TKQC
CREATE TABLE create_jobs (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  via_id        UUID REFERENCES via_accounts(id),
  status        VARCHAR(20) DEFAULT 'pending',  -- pending/running/done/stopped
  config        JSONB,         -- { bm_ids, count, template, currency, timezone, mode,
target_bms }
  total         INTEGER DEFAULT 0,
  ok_count      INTEGER DEFAULT 0,
  fail_count    INTEGER DEFAULT 0,
  created_at    TIMESTAMP DEFAULT NOW(),
  done_at       TIMESTAMP
);

-- Log từng TKQC
CREATE TABLE create_job_logs (
  id            BIGSERIAL PRIMARY KEY,
  job_id        UUID REFERENCES create_jobs(id) ON DELETE CASCADE,
  bm_id         VARCHAR(25),
  bm_name       VARCHAR(255),
  act_id        VARCHAR(25),
  act_name      VARCHAR(255),
  status        VARCHAR(30),   -- created / removed / shared_removed / bm_full / failed
  error         TEXT,
  created_at    TIMESTAMP DEFAULT NOW()
);
```

PHẦN 2 — SHARE TKQC (TDB)

Chia sẻ TKQC từ BM nguồn sang BM đích — 1 VIA chọn tay

2.1 Mô tả nghiệp vụ

Người dùng chọn 1 VIA, chọn danh sách TKQC (lấy từ BM của VIA hoặc import thủ công), chọn danh sách BM đích, cấu hình quyền, rồi hệ thống share từng TKQC vào từng BM đích. Mỗi cặp (TKQC × BM đích) là 1 tác vụ độc lập.

> Share TKQC là trao quyền truy cập — không phải chuyển sở hữu. TKQC vẫn thuộc BM nguồn. BM đích nhìn thấy TKQC đó dưới dạng "client ad account" với quyền trong phạm vi permitted_tasks được cấp.

2.2 Điều kiện tiên quyết

- VIA đã import, status = "ok", token còn hạn.
- TKQC nguồn: lấy từ bảng ad_accounts của VIA (status = Active = 1). TKQC status 2/101/201 bị lọc bỏ tự động.
- BM đích: nhập tay ID hoặc import CSV. Không cần là BM của VIA đang dùng.
- Quyền chia sẻ: chọn ít nhất 1 trong ADVERTISE / ANALYZE / MANAGE / DRAFT.

2.3 Luồng người dùng

① **Chọn VIA thực thi**
Dropdown chọn 1 VIA. Hệ thống load danh sách BM và TKQC của VIA từ DB. Hiện số lượng TKQC available.

② **Chọn TKQC nguồn**
Bảng TKQC với checkbox. Mặc định tất cả Active được chọn. TKQC Disabled/Unavailable/Closed tự động uncheck và disable. Hỗ trợ lọc theo tên/ID. Có thể import thêm từ CSV.

③ **Thêm BM đích**
Input nhập ID, hỗ trợ paste nhiều ID cách nhau bởi dấu phẩy/xuống dòng. Import CSV. Cảnh báo nếu BM đích trùng với BM nguồn.

④ **Cấu hình quyền**
Chọn mode: Global (1 bộ quyền tất cả) / Per-BM (mỗi BM đích có quyền riêng) / Per-TKQC (mỗi TKQC có quyền riêng). Xem chi tiết mục 2.4.

⑤ **Test 1×1 (khuyến nghị)**
Hệ thống share thử TKQC đầu tiên vào BM đích đầu tiên. Hiện kết quả: thời gian thực thi và quyền đã dùng. Nếu OK → "Token hợp lệ, có thể chạy hàng loạt".

⑥ **Xem ETA và xác nhận**
Hiện: tổng ops = TKQC × BM đích. Trừ đi ops đã done (từ done-set). ETA = effectiveOps × delayMs. User confirm trước khi chạy.

Tổng ops: tổng số lần gọi API sẽ thực hiện

EffectiveOps: Số cặp TKQCxBM target chưa thành công – tức là hệ thống sẽ thực hiện gọi API cho các cặp này

delayMs: thời gian cách nhau giữa 2 lần gọi API liên tiếp – tránh bị Fb chặn

⑦ **Chạy job**
Job chạy background. Frontend poll WebSocket mỗi 1.5s. Progress bar, per-BM stats, log realtime. Có thể Dừng → Resume sau.

2.4 Hệ thống phân quyền — 3 mode

Khi thực thi share, quyền được xác định theo thứ tự ưu tiên. Mức cao hơn override hoàn toàn mức thấp hơn:

Mode	Cách hoạt động	Ưu tiên	Khi nào dùng
Per-TKQC	Từng TKQC có bộ quyền riêng, áp dụng cho mọi BM đích nhận nó.	Cao nhất (1)	Kiểm soát chi tiết nhất. Phức tạp nhất.
Per-BM	Mỗi BM đích có quyền riêng. Mọi TKQC share vào BM đó đều dùng quyền đó.	Trung bình (2)	Agency có nhiều client với level quyền khác nhau.
Global	1 bộ quyền áp dụng đồng nhất cho tất cả cặp TKQC × BM.	Thấp nhất (fallback)	Đơn giản, đồng nhất. Mặc định của hệ thống.

> Logic resolvePerms: (1) acc.perBMPerms[bm.id] → (2) acc.customPerms → (3) bm.customPerms → (4) defaultPerms. Cấu trúc này đảm bảo luôn có quyền để gửi cho Facebook, không bao giờ gửi mảng rỗng.

Các quyền có thể cấp: ADVERTISE (chạy quảng cáo) — ANALYZE (xem báo cáo) — MANAGE (quyền đầy đủ, rủi ro cao, cẩn thận) — DRAFT (tạo nháp). Preset phổ biến nhất: ADVERTISE + ANALYZE.

2.5 Test 1×1

Test 1×1 là bước kiểm tra nhanh trước khi chạy hàng loạt. Hệ thống share thử 1 cặp (TKQC đầu tiên → BM đích đầu tiên) và trả về kết quả ngay.

Kết quả	Thông tin hiển thị
Thành công	"act_xxx → BM yyy — OK (312ms) · Quyền: ADVERTISE, ANALYZE". Gợi ý: token và quyền hợp lệ, có thể chạy toàn bộ.
Thất bại	Hiện error code + hint xử lý. Ví dụ: "[Code 190] Token hết hạn" hoặc "[Code 613] Rate limit — tăng delay lên 3000ms".

2.6 Cơ chế Resume — skipDone và done-set

done-set là tập hợp các cặp "accId:bmId" đã share thành công. Được persist vào DB sau mỗi batch. Khi job bị dừng và user chọn Resume:

- Backend bật skipDone = true.
- Với mỗi cặp (TKQC × BM đích): kiểm tra trong done-set. Nếu đã có → skip (tăng skipped counter, không gọi API).
- Chỉ share các cặp chưa có trong done-set.
- ETA tính từ effectiveOps = totalOps - alreadyDone.

> done-set được giới hạn 50K entries (LRU). Khi vượt quá: giữ 50K entry gần nhất, xóa cũ hơn. Đảm bảo không overflow DB với job rất lớn.

2.7 Logic engine (backend) — runBulkShare

```
token = decryptVIAToken(viaId)  // decrypt từ DB, không nhận từ frontend
doneSet = loadDoneSet(jobId)    // Set của "accId:bmId" đã done
totalOps = adAccounts.length × targetBMs.length

FOR acc IN adAccounts:
  IF shouldStop OR (stopOnError AND consecErrors >= maxConsecErrors): break

  // Tách BM: đã done (skip ngay) vs pending (cần share)
  pendingBMs = [], skippedBMs = []
  FOR bm IN targetBMs:
    IF skipDone AND doneSet.has(`${acc.id}:${bm.id}`):
      skippedBMs.push(bm)  // không gọi API
    ELSE: pendingBMs.push(bm)

  skipped += skippedBMs.length

  // Xử lý pendingBMs theo batch (parallelism 1-5)
  FOR batch OF chunk(pendingBMs, parallelism):
    IF shouldStop: break

    results = await Promise.allSettled(batch.map(bm => {
      perms = resolvePerms(acc, bm, defaultPerms)  // 4 cấp ưu tiên
      return withRetry(() => shareAdAccount(acc.id, bm.id, perms, token))
    }))

    FOR EACH result, bm IN zip(results, batch):
      IF success:
        successCount++; consecErrors = 0
        doneSet.add(`${acc.id}:${bm.id}`)
      ELSE:
        failCount++; consecErrors++
      completed++
      writeLog({ acc, bm, status, perms, error, retries })
      emit WebSocket { completed, success, fail, skipped, total }

    persistDoneSet(jobId, doneSet)
    sleep(delayMs)
```

2.8 shareAdAccount — Facebook API

```
// Primary: permitted_tasks dạng JSON string
POST act_{rawId}/agencies
body = { business: targetBMId, permitted_tasks: JSON.stringify(perms) }

// Fallback khi lỗi code 3 / 100 / 200:
body = { business: targetBMId }
perms.forEach((p, i) => body[`permitted_tasks[${i}]`] = p)

// withRetry logic:
// - code 190 (token expired) → KHÔNG retry, throw ngay
// - code 613 (rate limit) → retry với backoff 60s/120s/180s
// - code 17/32 (throttle) → retry với backoff exponential
// - code 3/100/200 → fallback format, nếu vẫn fail → throw
```

2.9 Tham số config job

Tham số	Default	Mô tả
viaId	—	UUID VIA thực thi. Backend tự decrypt token. Frontend không gửi token.
adAccounts	—	[{ id, name, bmId, customPerms?, perBMPerms? }]. Chỉ status Active.

targetBMs	—	[{ id, name, customPerms? }].
defaultPerms	—	Quyền fallback khi acc/bm không có customPerms.
delayMs	1200	Delay ms giữa mỗi batch. Tăng lên 3000ms+ khi gặp lỗi 613.
maxRetries	3	Số lần retry mỗi call. Không retry lỗi 190.
skipDone	true	Bỏ qua cặp đã có trong done-set. Cơ sở của Resume.
parallelism	1	Số BM đích xử lý song song. Tối đa 5.
stopOnError	false	Dừng toàn job khi đạt maxConsecErrors lỗi liên tiếp.
maxConsecErrors	3	Ngưỡng lỗi liên tiếp kích hoạt stopOnError.

2.10 Status object emit qua WebSocket

```
{
  state:       "running" | "done" | "stopped",
  total:       Number,   // totalOps = accs × bms
  completed:   Number,   // đã xử lý (bao gồm skip)
  success:     Number,
  fail:        Number,
  skipped:     Number,   // đã có trong done-set
  startedAt:   ISO8601,
  currentOp:   String,   // "act_xxx → BM yyy"
  perBMStats:  {
    [bmId]: { id, name, success, fail, skipped }
  }
}
```

2.11 DB Schema bổ sung

```
CREATE TABLE share_jobs (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  via_id          UUID REFERENCES via_accounts(id),
  status          VARCHAR(20) DEFAULT 'pending',
  config          JSONB,
  total_ops       INTEGER DEFAULT 0,
  completed       INTEGER DEFAULT 0,
  success_count   INTEGER DEFAULT 0,
  fail_count      INTEGER DEFAULT 0,
  skipped_count   INTEGER DEFAULT 0,
  per_bm_stats    JSONB,
  created_at      TIMESTAMP DEFAULT NOW(),
  started_at      TIMESTAMP,
  done_at         TIMESTAMP
);

-- done-set: persist cặp đã share thành công để Resume
CREATE TABLE share_done_keys (
  job_id    UUID REFERENCES share_jobs(id) ON DELETE CASCADE,
  acc_id    VARCHAR(30),
  bm_id     VARCHAR(25),
  done_at   TIMESTAMP DEFAULT NOW(),
  PRIMARY KEY (job_id, acc_id, bm_id)
);

CREATE TABLE share_job_logs (
  id            BIGSERIAL PRIMARY KEY,
  job_id        UUID REFERENCES share_jobs(id) ON DELETE CASCADE,
  acc_id        VARCHAR(30),
  acc_name      VARCHAR(255),
  source_bm_id  VARCHAR(25),
  target_bm_id  VARCHAR(25),
  target_bm_name VARCHAR(255),
  permissions   JSONB,
  status        VARCHAR(10),  -- "ok" | "fail" | "skip"
```

```
  error_code    INTEGER,
  error_message TEXT,
  retries       INTEGER DEFAULT 0,
  ts            TIMESTAMP DEFAULT NOW()
);
```

2.12 Xử lý lỗi Facebook API

STT	Ý nghĩa	Hành động hệ thống
1	Token thiếu quyền business_management	Không retry. Fail entry. Toast: Hãy cấp quyền đủ cho token.
2	Rate limit cấp ứng dụng	Retry với backoff. Toast: cảnh báo.
3	BM ID không hợp lệ hoặc tham số sai	Không retry. Fail entry. Highlight BM trong UI. Toast: BM ID không hợp lệ.
4	Token hết hạn	Không retry — token chết thì retry vô ích. Dừng job, cập nhật VIA. Toast: Token hết hạn, vui lòng cập nhật lại Token.
5	Token không phải admin BM nguồn	Không retry. Fail entry. Toast: Hãy dùng đúng Token admin BM đó.
6	Rate limit quá nhiều request	Retry backoff 60s/120s/180s. Auto tăng delay.

PHẦN 3 — KHÁNG NGHỊ TKQC

Submit appeal cho TKQC bị disabled — Internal Facebook API

3.1 Mô tả nghiệp vụ

Khi TKQC bị Facebook vô hiệu hóa (status = 2) hoặc đang chờ review (status = 100), người dùng có thể gửi kháng nghị để yêu cầu Facebook xem xét lại. Hệ thống tự động hóa việc gửi hàng loạt với nội dung template.

> Facebook không có endpoint kháng nghị qua Graph API công khai. Hệ thống dùng internal endpoint /ajax/help/contact/submit/request_ads_account_review — gọi từ tab trình duyệt đang đăng nhập Facebook, với cookie session thật. Đây là endpoint không chính thức, kết quả phụ thuộc hoàn toàn vào Facebook.

3.2 Điều kiện TKQC có thể kháng nghị

Status	Label	canAppeal	Ghi chú
2	**Disabled**	✓	Bị vô hiệu hóa. Xem disable_reason để chọn template phù hợp.
100	**Pending Risk Review**	✓	Chờ review rủi ro. Kháng để đẩy nhanh.
1	Active	✗	Đang hoạt động, không cần kháng.
201	Closed	✗	Đóng vĩnh viễn. Disable checkbox, không cho gửi.

3.3 Mã lý do disable (disable_reason)

Code	Tên lý do	Template khuyến nghị
0	**Không rõ lý do**	Default
1	**ADS_INTEGRITY_POLICY**	Policy Violation
2	**ADS_IP_REVIEW**	Default
3	**RISK_PAYMENT**	Default — lưu ý: cần xử lý payment trước khi gửi.
6	**BUSINESS_INTEGRITY_RAR**	Policy Violation
7	**PERMANENT_CLOSE**	Không gửi được. canAppeal = false.
9	**UNUSED_ACCOUNT**	Unusual Activity

3.4 Luồng người dùng

① **Chọn VIA thực thi**
Dropdown chọn 1 VIA. Cần VIA đang đăng nhập Facebook trên tab trình duyệt — internal API dùng cookie của tab đó.

② **Fetch TKQC bị disabled**
Hệ thống gọi API lấy owned_ad_accounts với fields account_status + disable_reason. Lọc canAppeal = true. Dedup theo actId. Sort: Disabled/Pending lên đầu.

③ **Chọn TKQC và template**

Checkbox chọn TKQC cần kháng. TKQC canAppeal = false bị disable. Chọn template (Default / Policy Violation / Unusual Activity / Custom). Xem trước nội dung với {act_id} đã replace.

④ **Cấu hình và chạy**
Cấu hình delay (mặc định 5000ms — cao hơn Share vì endpoint nhạy cảm hơn). Bật/tắt auto-retry khi bị rate limit. Confirm rồi chạy.

⑤ **Kết quả**
Log từng TKQC: sent / sent_retry / failed / skipped. Tổng kết OK/fail. Export CSV. Hiện cảnh báo: "Gửi OK không đảm bảo phục hồi — phụ thuộc xét duyệt của Facebook".

3.5 Cơ chế gửi appeal — Internal API

Backend inject script vào tab Facebook đang mở. Script chạy trong môi trường browser (world: MAIN) với đầy đủ cookie session:

```
// Bước 1: Đọc fb_dtsg từ HTML của tab Facebook
// Pattern 1: "DTSGInitialData",[],"token":"..."
// Pattern 2: "dtsg":{"token":"..."
// fb_dtsg là CSRF token bắt buộc cho mọi POST internal

// Bước 2: Lấy c_user từ document.cookie
// c_user = UID người dùng đang đăng nhập

// Bước 3: POST internal endpoint
POST /ajax/help/contact/submit/request_ads_account_review
Content-Type: application/x-www-form-urlencoded
credentials: include
body = {
  fb_dtsg:       <từ HTML>,
  appeal_reason: <template với {act_id} đã replace>,
  account_id:    <actId không có prefix "act_">,
  __user:        <c_user từ cookie>,
  __a:           "1"
}

// Xử lý response:
// HTTP 429       → rateLimited = true
// response chứa "errorSummary" hoặc "error" → fail
// Còn lại        → ok
```

> Tại sao inject qua tab trình duyệt? Endpoint nội bộ Facebook kiểm tra cookie session thật. Backend server không có cookie người dùng. Injection trong tab đang đăng nhập cho phép request gửi với đầy đủ cookie, giống như người dùng tự bấm trên giao diện Facebook.

3.6 Logic engine (backend)

```
FOR acc IN selectedAccounts:
  IF shouldStop: break

  content = template.replace(/{act_id}/g, acc.id)

  resp = await appealAdAccount({ adAccountId, content, token })

  IF resp.ok:
    log status = "sent"

  ELSE IF resp.rateLimited AND autoRetry:
    sleep(30000)  // chờ 30 giây
```

```
    retry = await appealAdAccount(...)
    log status = retry.ok ? "sent_retry" : "failed"

  ELSE IF resp.skip:
    log status = "skipped"

  ELSE:
    log status = "failed"

  sleep(delayMs)  // mặc định 5000ms
```

3.7 Trạng thái kết quả per-TKQC

status	Mô tả
`sent`	Gửi thành công lần đầu.
`sent_retry`	Gửi thành công sau khi retry (lần đầu bị rate limit, chờ 30s).
`skipped`	TKQC không đủ điều kiện kháng (canAppeal = false).
`failed`	Gửi thất bại — xem error message.

> Gửi appeal thành công (status = "sent") không đảm bảo TKQC được phục hồi. Kết quả do Facebook quyết định, phản hồi thường qua email trong 1–3 ngày. Cần fetch lại account_status sau đó để kiểm tra.

3.8 DB Schema bổ sung

```
CREATE TABLE appeal_jobs (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  via_id        UUID REFERENCES via_accounts(id),
  status        VARCHAR(20) DEFAULT 'pending',
  template_key  VARCHAR(30),  -- default / policy / unusual / custom
  template_text TEXT,
  delay_ms      INTEGER DEFAULT 5000,
  total         INTEGER DEFAULT 0,
  ok_count      INTEGER DEFAULT 0,
  fail_count    INTEGER DEFAULT 0,
  skip_count    INTEGER DEFAULT 0,
  created_at    TIMESTAMP DEFAULT NOW(),
  done_at       TIMESTAMP
);

CREATE TABLE appeal_job_logs (
  id        BIGSERIAL PRIMARY KEY,
  job_id    UUID REFERENCES appeal_jobs(id) ON DELETE CASCADE,
  act_id    VARCHAR(30),
  act_name  VARCHAR(255),
  status    VARCHAR(15),  -- sent / sent_retry / skipped / failed
  error     TEXT,
  ts        TIMESTAMP DEFAULT NOW()
);
```

PHẦN 4 — BACKEND API ENDPOINTS

REST API cho cả 3 sub-module

4.1 Tạo TKQC

Method	Path	Body / Query	Response
POST	/api/tkqc/create/scan-bm	{ viaId, bmIds[] }	{ results: [{ bmId, ownedCount, creationLimit, verification }] }
POST	/api/tkqc/create/start	{ viaId, bmIds[], countPerBM, nameTemplate, currency, timezoneId, mode, targetBMs? }	{ jobId }
GET	/api/tkqc/create/:jobId/status	—	{ state, completed, ok, fail, total, currentOp }
GET	/api/tkqc/create/:jobId/logs	—	{ logs: [{ bmId, actId, actName, status, error }] }
POST	/api/tkqc/create/:jobId/stop	—	{ ok }
GET	/api/tkqc/create/:jobId/export	?format=csv	File download

4.2 Share TKQC

Method	Path	Body	Response
POST	/api/tkqc/share/start	{ viaId, adAccounts[], targetBMs[], defaultPerms[], delayMs, maxRetries, skipDone, parallelism, stopOnError }	{ jobId }
POST	/api/tkqc/share/test	{ viaId, accId, bmId, perms[] }	{ ok, elapsedMs, perms, error? }
GET	/api/tkqc/share/:jobId/status	—	Status object (xem 2.10)
GET	/api/tkqc/share/:jobId/done-count	?accIds&bmIds (query)	{ count } — tính ETA
POST	/api/tkqc/share/:jobId/stop	—	{ ok }
POST	/api/tkqc/share/:jobId/resume	—	{ ok } — tự bật skipDone
GET	/api/tkqc/share/:jobId/logs	?filter=ok|fail|all	{ logs: [{ accId, bmId, status, perms, error, ts }] }
GET	/api/tkqc/share/:jobId/export	?format=csv	File download

4.3 Kháng Nghị TKQC

Method	Path	Body / Query	Response
GET	/api/tkqc/appeal/disabled	?viaId=uuid	{ accounts: [{ id, name, accountStatus, disableReason, canAppeal, bmId, bmName }] }
POST	/api/tkqc/appeal/start	{ viaId, accountIds[], templateKey, customText?, delayMs, autoRetry }	{ jobId }
GET	/api/tkqc/appeal/:jobId/status	—	{ state, total, ok, fail, skip, currentOp }

POST	/api/tkqc/appeal/:jobId/stop	—	{ ok }
GET	/api/tkqc/appeal/:jobId/logs	—	{ logs: [{ actId, status, error, ts }] }
GET	/api/tkqc/appeal/:jobId/export	?format=csv	File download

PHẦN 5 — FRONTEND VUE 3

Cấu trúc component và state cho module Tác vụ TKQC

5.1 Cấu trúc views và components

```
views/
  TacVuTKQC/                      // route: /tac-vu/tkqc
    index.vue                     // layout + tab: Tạo | Share | Kháng
    TaoTKQC.vue                   // sub-view Tạo TKQC
    ShareTKQC.vue                 // sub-view Share TKQC
    KhangTKQC.vue                 // sub-view Kháng Nghị

components/tkqc/
    VIASelector.vue               // Dropdown chọn VIA (dùng chung 3 sub-view)
    BMSelector.vue                // Chọn + hiển thị danh sách BM
    AccountTable.vue              // Bảng TKQC với checkbox, filter, status tag
    PermGrid.vue                  // Global / Per-BM / Per-TKQC perm selector
    JobProgress.vue               // Progress bar + log realtime (WebSocket)
    JobDoneCard.vue               // Kết quả cuối + export CSV
    PerBMStats.vue                // Bảng thống kê per-BM đích

composables/
    useShareJob.js                // start, stop, resume, poll status
    useCreateJob.js               // start, stop, poll create status
    useAppealJob.js               // start, stop, poll appeal status
    useJobPoller.js               // WebSocket + HTTP fallback (dùng chung)
    usePermMode.js                // resolvePerms, Global/Per-BM/Per-TKQC logic
    useETA.js                     // tính ETA từ effectiveOps × delayMs
```

5.2 Pinia store — useShareStore

```
export const useShareStore = defineStore('share', {
  state: () => ({
    // Input
    selectedViaId:   null,        // UUID VIA thực thi
    sourceBMs:       [],          // [{ id, name }]
    targetBMs:       [],          // [{ id, name, customPerms? }]
    adAccounts:      [],          // [{ id, name, bmId, bmName, account_status,
                                  //    customPerms?, perBMPerms? }]
    uncheckedAccIds: new Set(),   // IDs đã uncheck
    permMode:        "global",    // "global" | "per-bm" | "per"
    globalPerms:     ["ADVERTISE", "ANALYZE"],

    // Job
    activeJobId:  null,
    jobStatus:    null,           // object từ WebSocket
    jobLog:       [],
    isRunning:    false,
  }),
  getters: {
    // Chỉ lấy TKQC được chọn VÀ có status hợp lệ
    checkedAccounts: (s) => s.adAccounts.filter(a =>
      !s.uncheckedAccIds.has(a.id) &&
      ![2, 101, 201].includes(Number(a.account_status))
    ),
    totalOps: (s) => s.checkedAccounts.length * s.targetBMs.length,
  }
})
```

5.3 useJobPoller — WebSocket + fallback

```
export function useJobPoller(jobId, onUpdate, { interval = 1500 } = {}) {
  let ws = null, timer = null
```

```
  function startWS() {
    ws = new WebSocket(`ws://host/ws/job/${jobId}`)
    ws.onmessage = (e) => {
      const data = JSON.parse(e.data)
      onUpdate(data)
      if (data.state !== "running") stop()
    }
    ws.onerror = () => { ws = null; startPolling() }  // fallback HTTP
  }

  function startPolling() {
    timer = setInterval(async () => {
      const { data } = await api.get(`/tkqc/share/${jobId}/status`)
      onUpdate(data)
      if (data.state !== "running") stop()
    }, interval)
  }

  function stop() { ws?.close(); clearInterval(timer) }

  startWS()
  onUnmounted(stop)
  return { stop }
}
```

5.4 useETA — tính ETA trước khi chạy

```
export async function computeETA(jobId, accIds, bmIds, delayMs, skipDone) {
  const totalOps = accIds.length * bmIds.length
  let effectiveOps = totalOps

  if (skipDone && jobId) {
    const { data } = await api.get(`/tkqc/share/${jobId}/done-count`, {
      params: { accIds: accIds.join(","), bmIds: bmIds.join(",") }
    })
    effectiveOps = Math.max(0, totalOps - data.count)
  }

  const etaSec = Math.round(effectiveOps * delayMs / 1000)
  const etaStr = etaSec < 60   ? `${etaSec} giây`
               : etaSec < 3600 ? `${Math.round(etaSec/60)} phút`
               :                 `${(etaSec/3600).toFixed(1)} giờ`

  return { totalOps, effectiveOps, alreadyDone: totalOps - effectiveOps, etaStr }
}
```

PHẦN 6 — TRÌNH TỰ BUILD ĐỀ XUẤT

Từ core ra feature — ưu tiên unblock sớm nhất

Sprint 1 — Infrastructure (nền)

- Tạo bảng share_jobs, share_done_keys, share_job_logs, create_jobs, create_job_logs, appeal_jobs, appeal_job_logs.
- Implement graphPost helper với fallback format (JSON string → array-index).
- Implement withRetry với phân loại lỗi: auth không retry, rate limit backoff 60/120/180s.
- WebSocket server: ws://host/ws/job/:jobId — emit status updates.
- useJobPoller composable: WS + HTTP fallback.

Sprint 2 — Share TKQC (core)

- POST /api/tkqc/share/start → runBulkShare engine.
- resolvePerms function (4 cấp ưu tiên).
- done-set persist + getDoneCount + computeETA.
- stop + resume.
- POST /api/tkqc/share/test — Test 1×1.
- Frontend: ShareTKQC.vue với flow 7 bước, PermGrid, JobProgress, per-BM stats.

Sprint 3 — Tạo TKQC

- Scan BM API: ownedCount + creationLimit.
- POST /api/tkqc/create/start — 3 mode (Giữ / Đá / Share rồi đá).
- Template name resolver với 6 biến.
- Frontend: TaoTKQC.vue — preview tên, confirm, progress.

Sprint 4 — Kháng Nghị TKQC

- GET /api/tkqc/appeal/disabled — fetch và phân loại canAppeal.
- POST /api/tkqc/appeal/start — inject + internal API.
- Frontend: KhangTKQC.vue — danh sách TKQC disabled, 4 template, progress.

Sprint 5 — Polish

- Export CSV cho cả 3 sub-module.
- Log viewer với filter OK/Fail/All.
- Persist job history — xem lại lịch sử các job đã chạy.
- VIA health check trước khi bắt đầu job.